The Last Stand Game

Technical Design & Architecture Paper

1. Introduction

This document provides a comprehensive technical overview of a 2D top-down action game implemented in Java using the Swing framework. The project demonstrates core game engine concepts including entity management, AI pathfinding, collision detection, animation systems, combat mechanics, and wave-based gameplay.

The game features a player-controlled character that navigates a tiled environment, battles waves of enemies, collects powerups, and progresses through escalating difficulty. The codebase is organized around an entity hierarchy with shared behavior abstracted into base classes.

2. System Architecture

2.1 Class Hierarchy

The project follows an object-oriented inheritance model centered on a shared Entity base class:

Class	Extends	Role
Entity	GameObject	Base class — position, size, sprite, animation state
Player	Entity	Player character — input, movement, shooting, lives
Enemy	Entity (abstract)	AI enemy — pathfinding, combat, respawn logic
Obstacle	GameObject	Static map objects — collision bounds only
Bullet	GameObject	Projectile — direction, movement, lifetime
Powerup	GameObject	Collectible — base class for all powerup types
HealPowerUp	Powerup	Instant heal on collection

2.2 Core Design Principles

- Static sprite arrays shared across all instances of the same class to avoid redundant image loading
- Abstract Enemy class enforces subclass implementation while providing full AI behavior by default
- Separation of concerns: pathfinding, animation, combat, and movement are each handled in isolated methods
- Depth sorting via feet-position ensures correct visual layering without a Z-buffer

3. Player System

3.1 Input & Movement

The player is controlled using arrow keys for movement and WASD keys for shooting, allowing simultaneous movement and combat in any combination of 8 directions. Each frame the update() method reads the full set of held keys and constructs a direction vector:

```
if (movingUp)    dy -= 1;
f (movingDown)  dy += 1;
if (movingLeft)  dx -= 1;
if (movingRight) dx += 1;
```

This raw direction vector is then normalized to prevent diagonal movement from being faster than cardinal movement:

```
double length = Math.sqrt(dx * dx + dy * dy);
dx = (dx / length) * speed;
dy = (dy / length) * speed;
```

3.2 Sub-Pixel Accumulation

After normalization, the player's speed at a diagonal is approximately 2.12 pixels per frame (speed 3 / sqrt(2)). Since pixels are integers, fractional movement is accumulated across frames to prevent distance loss:

```
remainderX += dx;
int moveX = (int) remainderX;
remainderX -= moveX;
```

This ensures that over time the player travels the mathematically correct distance regardless of direction, producing smooth and consistent movement.

3.3 Collision Detection

The player uses a simplified foot-only hitbox deliberately smaller than the sprite. This is a common design decision that makes the game feel more forgiving:

```
return new Rectangle(x + 8, y + (height - 12), width - 16, 12);
```

Obstacle collision uses a save-and-restore pattern. The player's position before movement is saved, the move is applied, and if the new position intersects any obstacle the player is snapped back entirely:

int oldX = this.x;

int oldY = this.y;

// ... movement applied ...

if (getBounds().intersects(obs.getBounds())) { x = oldX; y = oldY; }

3.4 Shooting System

Bullets are fired in 8 directions matching the player's aim keys. A fire rate limiter prevents spamming:

Keys Held	Bullet Direction	Attack Sprite
W	NORTH	atk_up1.png
S	SOUTH	atk_down1.png
A	WEST	atk_left1.png
D	EAST	atk_right1.png
W + D	NORTHEAST	atk_upRight1.png
W + A	NORTHWEST	atk_upLeft1.png
S + D	SOUTHEAST	atk_downRight1.png
S + A	SOUTHWEST	atk_downLeft1.png

After firing, a canFire flag is set to false and only resets after fireRate milliseconds (default 500ms), preventing the player from firing more than twice per second.

4. Enemy System

4.1 State Machine

Each enemy operates on a two-state finite state machine:

State	Condition	Behavior
WALK	Distance to player > 55px	Navigate toward player using A* path
ATTACK	Distance to player <= 55px	Play attack animation, deal damage on frame 2

Transitions are checked every frame in updateState(). When switching from WALK to ATTACK the animation resets to frame 0. When the attack animation completes and the enemy is no longer in range it transitions back to WALK.

4.2 A* Pathfinding

The enemy uses the A* algorithm to navigate around obstacles toward the player. Rather than operating on individual pixels, the game world is divided into a 16x16 pixel grid, dramatically reducing the search space.

Grid Construction

The panel dimensions are divided by the cell size (16px) to produce a rows x cols boolean grid. Each obstacle's bounds are converted to grid coordinates and marked as blocked. To account for the enemy's 80x80 body width, obstacle cells are inflated outward by a radius calculated from the enemy's size:

```
int inflate = (Math.max(width, height) / 2) / CELL + 1;  // = 3 cells for 80px enemy
```

The start (enemy) and goal (player) cells are force-cleared after inflation to ensure they are never accidentally blocked by the expanded obstacle boundaries.

Search Algorithm

A* uses a min-heap priority queue ordered by f = g + h, where g is the actual cost from start and h is the octile distance heuristic to the goal. Eight-directional movement is supported with straight moves costing 10 and diagonal moves costing 14, approximating the true diagonal distance of sqrt(2) * 10 = 14.14.

Diagonal corner cutting is explicitly prevented: if either of the two cells adjacent to a diagonal move are blocked, that move is skipped. This prevents enemies from visually clipping through wall corners.

Path Caching

Recalculating A* every frame would be expensive. The path is cached and reused across frames, only recalculated when:

- The path cache cooldown (60 frames) expires
- The path is empty
- The player has moved more than 45px from the last known position (MOVE_THRESHOLD)

4.3 Movement & Wall Sliding

Each frame, followPath() moves the enemy one step toward the next waypoint. The direction to the waypoint is normalized and multiplied by speed to ensure consistent movement regardless of distance. If the direct move is blocked by an obstacle, two fallback options are attempted in order: horizontal-only movement, then vertical-only movement. This produces smooth wall-sliding behavior:

```
if      (clear at newX, newY) { x = newX; y = newY; }  // full move
else if (clear at newX, y  ) { x = newX; }             // slide horizontally
else if (clear at x,    newY) { y = newY; }            // slide vertically
        else  { stuckTimer += 5; }       // fully blocked
```

4.4 Stuck Detection

If the enemy's position doesn't change for 30 consecutive frames, stuck detection fires. It clears the current path, resets the pathfinding cooldown to force an immediate recalculation, and applies a random nudge of up to 2 cells in a random direction to break the enemy free from geometry traps.

4.5 Combat & Miss Chance

When in ATTACK state, the attack animation advances one frame every 10 ticks. Damage is evaluated on frame index 2 of the animation. At that point a 50% miss chance is rolled using a random number generator:

```
if (RNG.nextInt(100) >= MISS_CHANCE) {
        isDamageFrame = true;   // hit — deal damage
} else {
        showingMiss = true;     // miss — display MISS overlay
}
```

On a miss, a floating MISS image is rendered above the enemy with a fade-out alpha effect over 800 milliseconds, providing clear visual feedback to the player.

4.6 Contact Damage

Separate from the attack animation, enemies can deal touch damage when overlapping the player. A contact cooldown (CONTACT_COOLDOWN_MAX = 90 frames = ~1.5 seconds at 60fps) prevents this from firing every frame. This gives the player a brief recovery window after being hit.

5. Rendering System

5.1 Depth Sorting

To create the illusion of depth in a top-down view, all drawable objects are sorted by their feet position (Y + height) before rendering each frame. Objects with feet higher on screen are drawn first and appear behind objects with feet lower on screen:

```
drawables.sort((a, b) -> Integer.compare(
a.getY() + a.getHeight(),
b.getY() + b.getHeight()
));
```

This means a player standing below a tree will appear in front of it, while a player standing above the same tree will appear behind it — matching natural perspective expectations without any Z-buffer or layer system.

5.2 Animation System

Both Player and Enemy use a tick-based frame advancement system. An animationTick counter increments each frame and advances the sprite frameIndex when it reaches the configured speed threshold. This decouples animation speed from the game's frame rate.

The Player additionally tracks which sprite strip was active last frame. If the direction changes, the animation resets to frame 0 to prevent mid-stride visual glitches when the player turns.

6. Constants Reference

Constant	Value	Description
ATTACK_RANGE	55px	Distance at which enemy switches to ATTACK state
CELL	16px	A* grid cell size
PATH_REFRESH	60 frames	Frames between path recalculations (~1 second)
MOVE_THRESHOLD	45px	Player displacement that forces immediate replan
CONTACT_COOLDOWN_MAX	90 frames	Frames between contact-damage hits (~1.5 seconds)
WALK_ANIM_SPEED	5 ticks	Ticks per walk animation frame
ATTACK_ANIM_SPEED	10 ticks	Ticks per attack animation frame

Constant	Value	Description
MISS_CHANCE	50%	Probability that an attack does no damage
MISS_DURATION	800ms	Duration of floating MISS text fade-out
ATTACK_DURATION	120ms	Duration attack effect sprite is displayed on player
ANIMATION_SPEED	8 ticks	Player walk animation ticks per frame
Fire Rate	500ms	Minimum time between player shots

7. Powerup System

Powerups are collectible objects placed on the map. When the player's hitbox intersects a powerup's bounds, it is collected. Two categories exist:

Type	Behavior	Storage
HealPowerUp	Instantly restores one life	Not stored — applied immediately
Other Powerups	Applies a stat modifier (speed, fire rate, etc.)	Stored in currentPowerups map with a timestamp for duration tracking

Active timed powerups are applied each frame through the currentPowerups map, which pairs each Powerup object with the System.currentTimeMillis() timestamp of when it was collected, allowing expiry logic to be implemented by comparing elapsed time.

8. Wave System

The game progresses through waves of increasing difficulty. When a wave transitions, new obstacles are placed on the map. The system includes a player safety check: if the player's position overlaps a newly spawned obstacle, the player is relocated to a safe position before the wave begins. This prevents unfair instant-death scenarios from obstacles spawning on top of the player.

Enemies that are killed call respawn(), which teleports them to a random screen edge and fully resets their state, allowing them to re-enter the arena for the next wave.

9. Sound System

Audio is managed through a SoundManager singleton. Sound effects are triggered at specific game events:

Event	Sound File
Player walking	assets/music/walk.wav
Player shooting	assets/music/bow_and_arrow.wav
Enemy attack swing	assets/music/sword.wav

Walk sounds are triggered only when animationTick resets to 0, synchronizing the audio with the animation cycle rather than playing continuously while a key is held.

10. Summary

This project demonstrates a complete, self-contained 2D game engine built from first principles in Java. Key technical achievements include:

- Full A* pathfinding with obstacle inflation, corner-cut prevention, path caching, and stuck detection
- Normalized 8-directional movement for both player and enemy with consistent speed in all directions
- Sub-pixel accumulation for smooth fractional movement
- Foot-based depth sorting for correct visual layering without a Z-buffer
- Tick-based animation decoupled from frame rate
- Two-state enemy AI with contact cooldown, miss chance, and respawn system
- Fire rate limiting and 8-directional shooting with matching attack sprites
- Wave-based progression with player safety relocation on obstacle spawn

The architecture prioritizes clarity and separation of concerns, with each system isolated into well-named methods and configurable through named constants, making the codebase straightforward to extend with new enemy types, powerups, or game mechanics.